CASE STUDY

Agent Verification System

An autonomous, self-healing system that cross-verifies every newly activated real-estate agent against their public profile and the state's licensing authority — for all 50 U.S. states.

< 90 s	99.4 %	4 tiers	50 / 50
PER-AGENT VERIFICATION	TARGET FIRST-PASS RATE	BROWSER / HITL ROUTING	U.S. STATES COVERED

Contents

Next steps

Build date: 2026-05-18 · Repo: github.com/akashreboot/ak1 · Live demo: localhost:8501

TL;DR

Onboarding hundreds of agents per week across 50 states means hundreds of manual license verifications — each state's DRE has its own form, layout, and anti-bot posture. We prototyped a system that does the full verification in **under 90 seconds per agent**, with **graceful degradation** when state DRE sites change, and **clear HITL escalation** for cases that genuinely need a person. The prototype drives real onereal.com profiles and real state DRE sites end-to-end, and writes a SQLite ledger row for every run — no hardcoded 'success' placeholders.

The problem

What happens today	Pain
Manual checks per agent	Operator-hours scale linearly with hiring; one operator-hour per ~10 agents.
Site heterogeneity	Classic ASP, Salesforce Lightning, custom React, Drupal SPA — no reusable script across states.
Anti-bot defenses	Cloudflare and reCAPTCHA on some DRE sites get naive automation banned.
Disambiguation	A license number can map to multiple records (e.g. Notary + Real Estate Broker).
Silent breakage	A state site UI change can break the verifier with no signal until an audit.

Acute failure mode: an agent publishes publicly with an expired license that no human caught in time. Every state treats that as a compliance event.

What we built — 6-stage saga

Triggered by an agent.activated event from the CRM. Six durable activities run in sequence; each is idempotent and replayable.

```
1 · Fetch CRM metadata — what we believe about the agent
2 · Fetch onereal.com profile — what the agent publicly claims (Playwright)
3 · Cross-check CRM ↔ profile — name + state agreement
4 · Resolve adapter + browser — per-state YAML, tier-based routing
5 · Drive the DRE — fill license #, parse results,
disambiguate, navigate to detail,
extract expiration
6 · Compare expirations + decide — LLM classifier; write ledger / open HITL
```

Every step emits a trace span; every screenshot is persisted; every verdict lands in a SQLite ledger that Ops can audit.

Anti-bot tiering (T1 / T2 / T3 / T4)

Instead of using the heaviest tool for every state, the router picks the cheapest runner that still works. After 3 consecutive failures at a state's current tier, the router auto-promotes; after 3 consecutive successes at a promoted tier, it auto-demotes.

Tier	Runner	Cost	Used for
T1	Playwright Chromium	~$0.001 / call	~70% of states (no anti-bot)
T2	Camoufox (Firefox + C++ stealth)	~$0.003 / call	Cloudflare-light (~20%)
T3	Browserbase / Bright Data managed browsers	~$0.04 / call	Hard Cloudflare + CAPTCHA (~9%)
T4	Human-in-the-loop (Slack alert)	operator-minutes	Sites that resist T1–T3 (~1%)

Adapter-as-data — YAML, not code

Every state's DRE flow lives in a versioned YAML file. New state? Drop a YAML. Site redesign? Bump the version under a feature flag. Tier promotion? One-line config edit, not a redeploy.

```
state_code: WA
version: v2
anti_bot_tier: T2
dre:
search_url: https://professions.dol.wa.gov/s/license-lookup
flow: multi_page_detail
disambiguation_required: true
selectors:
license_input_label: License Number
license_input:
- input#License_Number
- lightning-input[data-label*='License'] input
detail_link_in_row:
- td:first-child a
detail_expiration:
- text=/Expiration Date:?\s*([A-Za-z]+ \d{1,2},? \d{4})/i
```

Disambiguation with confidence scoring

When a DRE returns multiple rows for the same license number (WA #141102 returns *both* 'Krista Cooper · Notary · Canceled' and 'James Bond NAM · Real Estate Broker · Active'), each row is scored:

Signal	Weight & method
License-type match	40% weight — fuzzy string match against expected type
Status = Active	20% — Canceled / Expired / Suspended penalized
Name similarity	30% — Jaccard on tokens between expected and row name
Geography hint	10% — row city ∈ agent service areas

Decision thresholds. Score ≥ 0.85 with margin ≥ 0.10 → confident pick. Score 0.55–0.85 → LLM tiebreak. Score < 0.55 → HITL.

Durable workflow semantics

Every step is replayable. A worker pod dying mid-verification does not lose progress — the saga resumes from the last successful activity. HITL pauses are durable signals: an operator's 'Approve' click resumes the workflow days later if needed.

Prototype: generator-based saga engine with Temporal-equivalent retry/replay/signal semantics. Production: Temporal Cloud or AWS Step Functions with Lambda workers.

What's real vs. production stand-ins

The prototype is honest about what's running for real and what is a swap-for-production stand-in with identical architecture.

Architecture says	Prototype runs
Temporal Cloud	In-process generator saga (same semantics)
Browserbase managed browser (T3)	Narrated as 'T3 simulated' in trace
Postgres RDS	SQLite at data/ledger.db (same SQL)
EventBridge + DynamoDB outbox	Streamlit button + JSON fixture
Slack incoming-webhook	In-app Slack-styled alerts with working buttons
Datadog metrics + traces	Streamlit dashboard reading the ledger
S3 artifact bucket	data/screenshots/ on local disk

Operational targets (year 1)

Metric	Target	How it's measured
Time per verification	< 90s p95	started_at → completed_at
First-pass match rate	≥ 97%	ledger.result='match' / total
HITL escalation rate	≤ 2.5%	open HITL items / total verifications
LLM spend per agent	< $0.005	Claude API metering
New-state adapter cost	< 1 engineer-day	git history of new .yaml files
MTTR after a state UI break	< 4 hours	adapter version bump + canary

Why this architecture

Alternatives we ruled out, with reasons:

Alternative	Why we said no
50 hand-written scrapers, no AI	Maintenance debt grows quadratically with site changes.
Pure-AI agents (Computer Use for every call)	~$0.50/call × 50K calls/yr = unjustifiable cost.
Agentic loop with a generic WebBrowser tool	No durability, no replay, no per-state cost control.
Selenium-only	Same fragility as Playwright + worse stealth + no AI fallback path.
Buy a compliance-as-a-service vendor	None cover the brokerage-specific cross-check between the public profile and state DRE.

The principle: deterministic selectors when the page is stable, AI fallback the moment it isn't — same workflow code path for both. We never choose between cheap and smart.

Risk register

Risk	Mitigation

State adds new anti-bot defense	Router auto-promotes tier; Adapter Registry shows red; ship new adapter version.
LLM model deprecation	Adapter declares the model; switching is a config edit.
reCAPTCHA appears on a previously-bypassed site	Workflow routes to T4 (HITL) automatically; operators paged via Slack.
CRM schema change	Single mapper module with explicit field-by-field test fixtures.
LLM hallucinates wrong field	Every AI-extracted value is regex/format-checked; failures fall through to HITL.

8-week rollout plan

Window	Deliverable
Week 1	Adapter YAMLs for the 10 highest-volume states
Week 2	Temporal Cloud / Step Functions deployment; EventBridge wired to CRM
Week 3	Browserbase contract for T3; first 5 states live in production
Week 4	Canary rollout to 10% of agent.activated events; Datadog dashboard live
Week 6	Full 50-state coverage; on-call rotation; runbook for adapter breakage
Week 8	Stage 2: outbound expiration-reminder pipeline driven by the same ledger

Next steps

- Decide on Temporal Cloud vs. AWS Step Functions (cost + ops profile).
- Sign Browserbase contract (T3 paths require it for hard CAPTCHA states).
- Confirm CRM event schema for agent.activated from Salesforce CDC.
- Identify the first 10 states by activation volume (likely TX, CA, FL, NY, WA, GA, NC, AZ, CO, NJ).
- Allocate one engineer-week per 10 adapters for the initial 50-state coverage.

Reference links

- Repository: github.com/akashreboot/ak1
- Playwright: playwright.dev/python
- Camoufox: github.com/daijro/camoufox
- Browserbase: browserbase.com
- Temporal: temporal.io
- Anthropic API: docs.claude.com

Three sentences to remember: (1) *It has to be cheap when nothing's wrong, and graceful when something is.* (2) *Adapter-as-data, not adapter-as-code.* (3) *AI is a fallback, not the headline.*